OWASP Juice Shop

Owner: DevSecOps project team
Reviewer:
Contributors: Yahya Bichiou, Mohamed Amine Bettaieb
Date Generated: Tue Nov 03 2026

Executive Summary

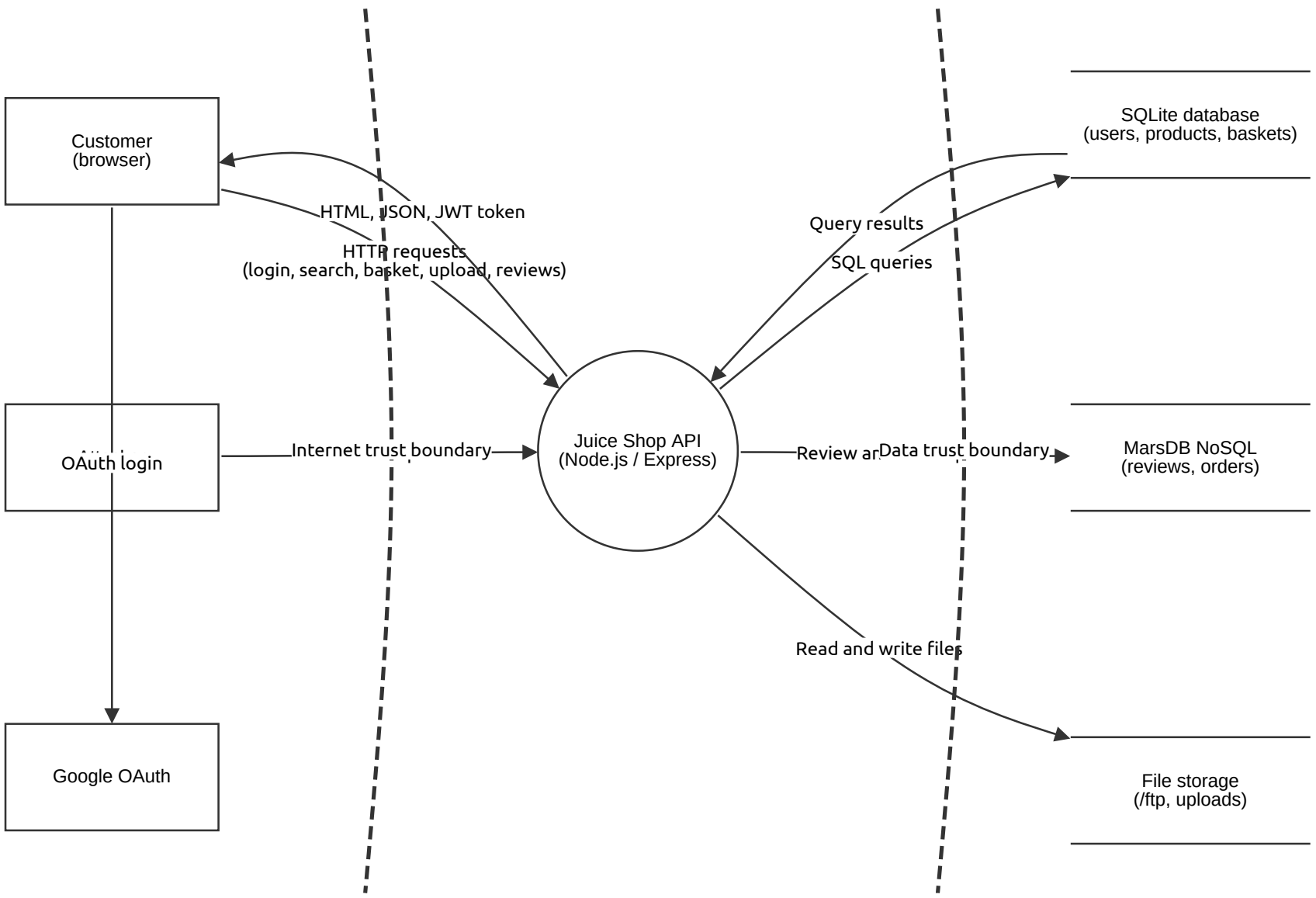
High level system description

E-commerce web app with an Angular frontend, a Node.js / Express API, a SQLite database and a MarsDB NoSQL store, shipped as a Docker image. Deliberately vulnerable. Target of our DevSecOps pipeline. Each threat names the pipeline stage that detects it.

Summary

Total Threats	26
Total Mitigated	0
Total Open	26
Open / Critical Severity	0
Open / High Severity	12
Open / Medium Severity	11
Open / Low Severity	3

Juice Shop architecture



Juice Shop architecture

Customer (browser) (Actor)

Description: Registered or anonymous shopper using the Angular single page app.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	Weak passwords and no MFA by default	Spoofing	Medium	Open		Customer accounts accept weak passwords and MFA is optional, so accounts can be taken over by guessing or credential stuffing. Detected by pipeline stage: DAST (OWASP ZAP) + code review	Enforce a password policy, check passwords against breached lists, encourage or require 2FA.
2	Customer can deny placing an order	Repudiation	Low	Open		Orders and payments are not tied to a tamper evident audit trail, so a customer can dispute an action and we cannot prove it. Detected by pipeline stage: Code review	Log order and payment events with user id, timestamp and request id to an append only log.

Attacker (Actor)

Description: Anonymous internet user sending crafted requests directly to the API.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Google OAuth (Actor)

Description: External identity provider used for 'Log in with Google'.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
3	OAuth accounts get a predictable password	Spoofing	High	Open		Users created through OAuth get a local password derived from their email address, so anyone who knows the email can log in with the normal form. Detected by pipeline stage: SAST (Semgrep / CodeQL) + code review	Do not create a local password for OAuth users, or generate a random one; validate the ID token server side.

Juice Shop API (Node.js / Express) (Process)

Description: Express server exposing the REST API and serving the Angular frontend. Runs in a Docker container.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
4	JWT forgery with a hardcoded signing key	Spoofing	High	Open		The RSA private key used to sign JWTs is hardcoded in lib/insecurity.ts. Anyone with the source can forge a token for any user, including the admin. Detected by pipeline stage: Secrets scan (Gitleaks, pre-commit) + SAST	Move the key to a secret manager or environment secret, rotate it, and verify the algorithm on every token (reject 'none').

Number	Title	Type	Severity	Status	Score	Description	Mitigations
5	IDOR on basket API	Elevation of privilege	High	Open		GET /rest/basket/:id returns any basket by id without checking it belongs to the logged in user. Detected by pipeline stage: Security unit tests	Check ownership server side: the basket id must match the id in the user's token.
6	Mass assignment on registration	Elevation of privilege	High	Open		POST /api/Users accepts a 'role' field, so a new user can register directly as admin. Detected by pipeline stage: Security unit tests + SAST	Whitelist accepted fields on registration and always set the role server side.
7	No audit log of sensitive actions	Repudiation	Medium	Open		Admin actions, login failures and payment changes are not recorded in a protected log, so attacks cannot be traced. Detected by pipeline stage: Code review	Structured security logging of authentication, authorization failures and admin actions, shipped to a central log store.
8	Vulnerable third party npm packages	Tampering	High	Open		The API depends on many npm packages, some with known CVEs. A vulnerable or compromised dependency runs with the app's privileges. Detected by pipeline stage: SCA (Trivy fs, npm audit, Dependabot) + SBOM (Syft)	Keep dependencies updated, pin versions with the lockfile, generate an SBOM, block builds with CRITICAL CVEs.
9	Container image with excess privileges or vulnerable OS packages	Elevation of privilege	Medium	Open		If the container runs as root or the base image has known CVEs, an RCE in the app gives the attacker more control over the host. Detected by pipeline stage: Dockerfile scan (Hadolint, Checkov) + image scan (Trivy) + signing (Cosign)	Run as a non-root USER, use a minimal base image, scan the image on every build, sign it.

SQLite database (users, products, baskets) (Store)

Description: Relational database accessed through Sequelize and some raw SQL queries.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
10	Passwords stored as unsalted MD5	Information disclosure	High	Open		User passwords are hashed with MD5 and no salt. After any SQL injection dump they are cracked in seconds. Detected by pipeline stage: SAST (Semgrep / CodeQL)	Hash passwords with bcrypt or Argon2 with a per user salt.
11	Database file not encrypted at rest	Information disclosure	Low	Open		The SQLite file sits unencrypted inside the container; anyone with access to the volume reads all data. Detected by pipeline stage: IaC / config review (Checkov)	Encrypt the volume or move to a managed database with encryption at rest and least privilege accounts.

MarsDB NoSQL (reviews, orders) (Store)

Description: In-memory MongoDB-like store for product reviews and orders.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
12	NoSQL injection modifies other users' reviews	Tampering	High	Open		PATCH /rest/products/reviews passes the request body straight into the update query, so operators like $ne update many reviews at once. Detected by pipeline stage: SAST (Semgrep / CodeQL)	Validate types (id must be a string), never pass raw request objects to queries, update by exact id only.
13	NoSQL $where injection blocks the server	Denial of service	Medium	Open		The product review lookup builds a $where expression from user input, so a payload like sleep() freezes the event loop. Detected by pipeline stage: SAST (Semgrep / CodeQL) + DAST	Remove $where queries, validate input, set query timeouts.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
14	Review posted under another user's name	Spoofing	Medium	Open		The review author is taken from the request body instead of the session, so a user can post as someone else. Detected by pipeline stage: Security unit tests	Take the author from the authenticated token, ignore it in the body.

File storage (/ftp, uploads) (Store)

Description: Server folders holding downloadable files (/ftp) and user uploads (complaints, profile images).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
15	/ftp folder exposes confidential files	Information disclosure	High	Open		The /ftp directory is browsable and contains internal documents and backup files such as package.json.bak. Detected by pipeline stage: DAST (OWASP ZAP)	Remove sensitive files, disable directory listing, require authentication for internal documents.
16	File type filter bypass with a poison null byte	Information disclosure	Medium	Open		The /ftp route only serves .md and .pdf, but an encoded null byte (%2500) bypasses the check and downloads any file. Detected by pipeline stage: DAST (OWASP ZAP) + SAST	Normalize and decode the path before checking it, use an allow list of exact file names.
17	Unrestricted file upload	Tampering	Medium	Open		Upload size and type are only checked in the frontend, so crafted uploads (oversized files, XML with external entities) reach the server. Detected by pipeline stage: SAST + DAST	Validate type, size and content server side, disable XML external entities in the parser.

HTTP requests (login, search, basket, upload, reviews) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
18	SQL injection bypasses login	Spoofing	High	Open		routes/login.ts builds the SQL query by concatenating email and password, so ' OR 1=1-- logs in as the first user (admin). Detected by pipeline stage: SAST (Semgrep / CodeQL) + security unit tests	Use parameterized queries (Sequelize replacements or the ORM), never string concatenation.
19	SQL injection in product search dumps the database	Information disclosure	High	Open		routes/search.ts injects the q parameter into a raw query, so a UNION SELECT returns the whole Users table. Detected by pipeline stage: SAST (Semgrep / CodeQL) + DAST	Parameterized query, input validation, least privilege database account.
20	Brute force on login, no rate limiting	Spoofing	Medium	Open		The login endpoint has no rate limit or lockout, so passwords can be guessed at high speed. Detected by pipeline stage: DAST (OWASP ZAP)	Rate limit by IP and account, progressive delays, lockout alerts.
21	Traffic not protected by TLS	Information disclosure	Medium	Open		The app is served over plain HTTP, so credentials and tokens can be sniffed on the network. Detected by pipeline stage: DAST (OWASP ZAP)	Terminate TLS in front of the app and send HSTS.

HTML, JSON, JWT token (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
22	Cross site scripting in search results	Tampering	High	Open		The frontend renders the search term with bypassSecurityTrustHtml, so injected script runs in the victim's browser and can steal the JWT from localStorage. Detected by pipeline stage: SAST (Semgrep / CodeQL) + DAST	Never bypass Angular sanitization, encode output, add a strict Content Security Policy.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
23	Error responses leak stack traces	Information disclosure	Medium	Open		Unhandled errors return the full stack trace and framework details to the client. Detected by pipeline stage: DAST (OWASP ZAP) + security unit tests	Generic error handler in production, log details server side only.
24	Missing security headers	Information disclosure	Low	Open		Responses lack a Content Security Policy and other hardening headers. Detected by pipeline stage: DAST (OWASP ZAP)	Add CSP, X-Content-Type-Options, Referrer-Policy and frame protections (helmet).

Malicious requests (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
25	Normal user reaches the admin section	Elevation of privilege	High	Open		The administration page is only hidden by a frontend route guard; the API calls behind it accept any logged in user or are guessable. Detected by pipeline stage: DAST + security unit tests	Enforce role checks on every admin API route server side.
26	No limit on request rate or size	Denial of service	Medium	Open		Any client can flood the API or send very large bodies, exhausting the single Node.js process. Detected by pipeline stage: DAST (OWASP ZAP)	Rate limiting, body size limits, run several instances behind a load balancer.

OAuth login (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

SQL queries (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Query results (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Review and order queries (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Read and write files (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations